UNIT NO: 2 : Managing Tables and Data, Data Control and Transaction Control Command

EXPLAIN CREATE COMMAND WITH EXAMPLE

- This command is used to create table.
- The CREATE table command is useful a new table in your present user.
- By using, we have to specify fieldname, data type and size.
- Each of the fields should be separated by comma.
- the SQL statement will be terminated with a semicolon.
- Create tables to store data by executing the SQL CREATE TABLE statement.
- This statement is one of the data definition language (DDL) statements.
- These statements have an immediate effect on the database, and they also record information in the data dictionary.

RULES FOR CREATING Table names and column names:

- Must begin with a letter
- Must be 1 to 30 characters long
- Must contain only A–Z, a–z, 0–9, _, $, and #
- Must not duplicate the name of another object owned by the same user
- Must not be an Oracle Server reserved word

SYNTAX :

CREATE TABLE <<table name>>
(<<column name>><Data type><Size> [CONSTRAINT], <<column name>><Data Type><Size> [CONSTRAINT],……);

In CREATE TABLE Statement:

<Table name>	Specify name of the table
<Column name>	Specify fieldname
<data type>	Specify the type of data that field hold
<size>	Specify length for field
[CONSTRAINT]	Specify name of constraint if any

EXAMPLE

CREATE TABLE student **(**rollno number(3),first_name varchar(20),last_name varchar(20), address varchar2(30),city varchar(10),pincode number(10)**)**;

EXPLAIN DDL STATEMENTS WITH SUITABLE EXAMPLE.

- DDL stands for Data Definition Language.
- The DDL commands are the set of SQL commands that can be used to create, modify or delete the database structure but not data.
  - **Following are the list of DDL commands:**
    a. **CREATE**
    b. **ALTER**
    c. **DROP**
    d. **RENAME**
    e. **TRUNCATE**

CREATE:

- This command is used to create table.

ALTER:

- By using alter table command we can modify the structure of the database.

- Using ALTER TABLE statement to:
- Add a new column
- Modify an existing column
- Define a default value for the new column
- Drop a column

SYNTAX
ADDING NEW COLUMNS
ALTER TABLE <table name> ADD (new<column name> data type (size));
MODIFYING EXISTING COLUMNS
ALTER TABLE <table name> MODIFY (<column name> new data type(new size));
DROPING EXISTING COLUMNS
ALTER TABLE <table name> DROP(<column name>);

In the syntax:

<Table name>	Specify name of the table
<Column name>	Specify fieldname
<data type>	Specify the type of data that field hold
<size>	Specify length for field

EXAMPLE
ADDING NEW COLUMNS
ALTER TABLE student ADD (country VARCHAR (10));
MODIFYING EXISTING COLUMNS
ALTER TABLE student MODIFY (last_name VARCHAR2 (30));
DROPING EXISTING COLUMNS
ALTER TABLE dept DROP(country);

DROP:

- This command is useful for removing the definition of a table from oracle database.
- When you drop a table, the database loses all the data in the table and all the indexes associated with it

SYNTAX
DROP TABLE <table name>; [purge]

RENAME:

- This command is used to rename a table, view, sequence, or synonym.

SYNTAX
RENAME <old name _table/view/synonym> TO <new name _table/view/synonym>;

In the syntax:

<old name>	Specify the old name of the table, view, sequence, or synonym
<new name>	Specify the new name of the table, view, sequence, or synonym

EXAMPLE
RENAME student TO stud;

TRUNCATE:

- The SQL TRUNCATE TABLE command is used to delete all the records from an existing table by reinitializing the table's structure.
- This command instructs the database to deallocate the space for all records in a table and change this table's structure by resetting the table size.

- This is the reason why it is deemed to be a Data Definition Language (DDL) operation rather than Data Manipulation Language (DML), even though all the table data is removed.
- Logically, the TRUNCATE TABLE statement performs similarly to the DELETE TABLE statement but without the WHERE clause. However, TRUNCATE is much faster than DELETE and does not allow roll back once committed.

SYNTAX
TRUNCATE TABLE <table_name>;
EXAMPLE
TRUNCATE TABLE student ;

EXPLAIN DML STATEMENTS WITH SUITABLE EXAMPLE.

- DML stands for Data Manipulation Language.
- The DML commands are the set of SQL commands that can be used to insert, modify or delete the records of the table.
- Following are the list of DDL commands:
  - **INSERT**
  - **UPDATE**
  - **DELETE**

INSERT:

- This command is used to insert a row in table.
- With the help of Insert into command we can enter the data into the table one record at a time.

SYNTAX
INSERT INTO <table name> (<column name>, <column name>) VALUES (expression, expression);
EXAMPLE
INSERT INTO student VALUES (1, 'avni','bhatt','1 narmada park','rajkot');

UPDATE:

- With the help of this command we can update the records of a table it means we can set the records of the tables.
- Update all the rows in a table at a time.
- In this type of condition, we must use where clause.

SYNTAX
UPDATE TABLE SET <column_name> = value [, <column_name> = value….] [WHERE <condition>];
EXAMPLE
UPDATE emp SET deptno = 30 WHERE empid=7369;

Delete:

- To delete one or more rows from a table, DELETE command is used.
- In this statement, First, you specify the name of the table from which you want to delete data. Second, you specify which row should be deleted by using the condition in the WHERE clause.
- If you omit the WHERE clause, the Oracle DELETE statement removes all rows from the table.
- Note that it is faster and more efficient to use the TRUNCATE TABLE statement to delete all rows from a large table.

SYNTAX :
DELETE FROM table_name WHERE condition;
EXAMPLE :
DELETE FROM student WHERE id = 1;

EXPLAIN SELECT STATEMENTS WITH SUITABLE EXAMPLE.

The SELECT statement is used to select data stored in your database. The data returned is stored in a result table, called the result-set.

SYNTAX

SELECT */<fieldname> |[DISTINCT <fieldname>] FROM <*table_name*>
[GROUP BY <fieldname> [HAVING <condition>]]
[ORDER BY <fieldname> [ORDER]];

In syntax

*****	Use to display all fields & records
<field name>	Specify fieldname to display some specific fields
DISTINCT	Used to remove repeated values from fields
<table name>	Specify name of the table to display
WHERE <condition>	Used to retrieve records that match some specific condition
GROUP BY	Used to perform function on group value
HAVING	Used to match conditional records with group function
ORDER BY	Arrange records in some specific order

EXAMPLE

SELECT deptno, dname FROM dept;
SELECT * FROM emp;

SELECT WITH CLAUSES:

- **SELECT ALL ROWS/COLUMNS FORM TABLE**
  SELECT * FROM DEPT;
- **SELECT SPECIFIC COLUMNS /ROWS FORM TABLE**
  SELECT ENAME, SAL, JOB FROM EMP;
- **REMOVE DUPLICATE ROWS**
  SELECT DISTINCT * FROM DEPT;
- **USE ARITHMETIC OPERATOR (+,_,*,/)**
  SELECT ENAME, SAL, SAL + 1000 FROM EMP;
- **USE RELATIONAL OPERATPR (>,<,>=,<=,=,<>)**
  SELECT * FROM EMP WHERE DEPTNO<>10;
- **USE LOGICAL OPERATOR (AND, OR, NOT)**
  SELECT ENAME, SAL, JOB FORM EMP WHERE DEPTNO>10 OR DEPTNO <30;
- **SORTING DATA WITH SELECT**
  SELECT * FROM EMP ORDER BY ENAME DESC;

EXPLAIN CONSTRAINTS.

- The Constraints are rules that define which data values are valid during insert, update & delete operations.
- In some situation, we need the data that will be check before it will enter into the table.
- Constraints enforce rules at the table level.
- Constraints prevent the deletion of a table if there are dependencies.
- Oracle server uses constraints to prevent invalid data entry into table.
- Constraints can be used to do following:
- Defines rules on data in a table when a row is inserted, updated or deleted from table.
- Prevent the deletion of table if there are dependencies from other table.
- Provide rules for Oracle tools such as oracle developers.

- Oracle gives permission to attach the Data Constraints in the table.
- Once the constraints attach in a table then any type of operation is automatically checked before data storage.
- When constraints are enabled then they are inserted it means when constraints are disabled then they are not inserted.
- Constraint may be defined either At the same time as the table is created Or After the table has been created.
- 4 types of constraints are mainly used:
    - **PRIMORY KEY CONSTRAINTS**
    - **FOREIGN KEY CONSTRAINTS**
    - **UNIQUE CONSTRAINTS**
    - **CHECK CONSTRAINTS**
    - **NOT NULL**

SYNTAX
CREATE TABLE <table_name> (<column_name> data type (<size>), Column_CONSTRAINTS],……[table_Constraint])

In syntax:

<Table name>	Specify name of the table
<Column name>	Specify fieldname
Data type	Specify Types of the data that field hold
Constraint	Specify constraint name

PRIMARY KEYCONSTRAINTS

- A primary key constraint creates a primary key for table.
- A primary key constraint is column or set of columns that uniquely identify each row in a table.
- A table can have only one primary key constraint but can have several unique constraints.
- Primary key is used to identify each row is unique or not in the table.
- The table in which primary key is defined that is called primary table.
- Primary key cannot contain NULL values as well as cannot contain duplicate entry.
- In Primary Key, LONG and LONG ROW data type cannot be allowed.
- Unique Index is created automatically on Primary key.
- It can apply at column level as well as table level.

SYNTAX:
(COLUMN LEVEL)
CREATE TABLE <<table name>> (<<column name>> <Data Type><Size> PRIMARY KEY)
(TABLE LEVEL)
CREATE TABLE <<table name>> (…<Column Definition>… PRIMARY KEY (<<column name>>, <<column name>>, <<column name>>…));
EXAMPLE:
(COLUMN LEVEL)
CREATE TABLE stud (studno number (10) PRIMARY KEY, address varchar(20), city varchar(10), mobile varchar(10));
(TABLE LEVEL)
CREATE TABLE stud (studno number(10), address varchar(20), city varchar(10), mobile varchar(10), PRIMARY KEY(studno, address, city mobile));

FOREIGN KEYCONSTRAINTS

- A FOREIGN key is a column or set of columns that refers to a primary key in the same table or another table.
- Foreign key is based on the data values and are purely logical.

- A foreign key value must match an existing primary key value or unique key value or else is Null.
- Foreign key must reference either a primary key or unique key column.
- Foreign key is a column that defines how tables related to each other.
- It is a single or group of columns whose values are derived from the Primary Key of some other table.
- The table in which foreign key is defined that is called a Foreign Table or Detail Table.
- When we use foreign key constraint, two tables are used like Parent Table and Child Table.
- Parent table is a main table and this type of table contains primary key and foreign key.
- Child table is suitable of parent table whose data values are checked for the current database only.
- This type of foreign key constraint is called Referential Integrity Constraint.
- Parent table can be reference in the foreign key by using the word REFERENCE.
- Child table contains duplicate and null values.

SYNTAX:
(COLUMN LEVEL)
CREATE TABLE <<table name>>(<<column name>> <Data Type><Size> REFERENCES <<table name>> (<<column name>>));
(TABLE LEVEL)
CREATE TABLE <<table name>> (<<column name>> <Data Type><Size>, FOREIGN KEY (<<column name>>) REFERENCES <<table name>>);
EXAMPLE:
(COLUMN LEVEL)
CREATE TABLE stud (studno number(10) REFERENCES student (student_no), address varchar(20), city varchar(10), mobile varchar(10));
(TABLE LEVEL)
CREATE TABLE stud (studno number(10), address varchar(20), city varchar(10), mobile varchar(10), FOREIGN KEY(student_no,student_name) REFERENCES student);

UNIQUE KEYCONSTRAINTS

- Unique key constraints require that every value in a column or set of columns be unique.
- That means no two rows of a table can have duplicate values in specified column or set of columns.
- The column included in the definition of unique key constraints is called unique key.
- If more than one column contains unique constraints than it is known as composite unique key.
- Unique constraints allow the input of NULL values but it doesn't allow duplicate value.
- A unique constraint can be defined at the column level or table level.
- Unique index is created automatically
- This constraint cannot support long & long row data type.

SYNTAX:
(COLUMN LEVEL)
CREATE TABLE <<table name>> (<<column name>> <Data Type><Size> UNIQUE);
(TABLE LEVEL)
CREATE TABLE <<table name>> (…<Column Definition>… UNIQUE (<<column name>>, <<column name>>, <<column name>>…));
EXAMPLE:
(COLUMN LEVEL)
CREATE TABLE stud (studno number(10) UNIQUE, address varchar(20), city varchar(10), mobile varchar(10));
(TABLE LEVEL)

CREATE TABLE stud (studno number(10), address varchar(20), city varchar(10), mobile varchar(10), UNIQUE (studno, address, city mobile));

CHECK CONSTRAINT

- Check constraints defines a condition that each row must satisfied.
- The condition can use the same as we specified in query condition.
- It checks the column in a table.
- It returns an output if the column is true otherwise it returns an error.
- It allows duplicate values.
- It does not allow NULL values.
- It can apply at column level as well as table level.

SYNTAX:
(COLUMN LEVEL)
CREATE TABLE <<table name>> (<<column name>> <Data Type><Size> CHECK (condition));
(TABLE LEVEL)
CREATE **TABLE <<table name>> (…<Column Definition>… CHECK (<<column name>> CONDITION));**
EXAMPLE: (TABLE LEVEL)
CREATE TABLE stud (empno number(10), empname varchar(20),mobile varchar(10), CHECK(empno BETWEEN 101 AND 300));
(COLUMN LEVEL)
CREATE TABLE stud (studno number(10) CHECK (studno BETWEEN 0 AND 100), mobile varchar(10));

NOT NULL CONSTRAINT

- A NOT NULL constraint protects one or more columns in a table.
- It allows duplicate values.
- It does not allow NULL values.
- It can be applied only at a column level.

SYNTAX
CREATE TABLE <<table name>> (<<column name>><Data Type><Size> NOT NULL, <<column name>><Data Type><Size>);
EXAMPLE
CREATE TABLE address (name varchar(20) NOT NULL, address varchar (30),city varchar(15) NOT NULL);

DEFAULT Constraint

The DEFAULT constraint is used to set a default value for a column.
The default value will be added to all new records, if no other value is specified.

SYNTAX
CREATE TABLE <<table name>> (<<column name>><Data Type><Size> DEFAULT <VALUE>, <<column name>><Data Type><Size>);
EXAMPLE:
CREATE TABLE Persons (ID int NOT NULL, LastName varchar(255) NOT NULL, FirstName varchar(255), Age int, City varchar(255) DEFAULT 'Rajkot');
EXAMPLE:
CREATE TABLE Orders (ID int NOT NULL, OrderNumber int NOT NULL, OrderDate date DEFAULT GETDATE());

EXPLAIN GROUP BY AND HAVING CALUSE WITH EXAMPLE.

Group By:
A GROUP BY clauses is useful to divide group of the data whenever we use a group function in SELECT statement to perform calculation with the group of fields.

The GROUP BY statement is often used with aggregate functions (COUNT(), MAX(), MIN(), SUM(), AVG()) to group the result-set by one or more columns.

SYNTAX
SELECT <<aggregate function>>/expre FROM <table name> WHERE <condition> GROUP BY <column1>,<column2>… [HAVING <condition>];
EXAMPLE
SELECT deptno,sum(sal) FROM emp GROUP BY deptno;
SELECT deptno,job,max(sal) FROM emp GROUP BY deptno,job;

Having:
Having clause is used to filter data according to the conditions provided.
Having clause is only used with the SELECT clause.
HAVING Clause is implemented in column operation. Having clause is generally used after GROUP BY.
The HAVING clause was added to SQL because the WHERE keyword cannot be used with aggregate functions.

EXAMPLE
SELECT deptno, avg(sal) FROM emp GROUP BY deptno HAVING avg(sal)>1000 ;

ROLLUP

The ROLLUP function enables a SELECT statement to calculate multiple levels of subtotals across a specified group of dimensions and a grand total.
The ROLLUP function is a simple extension to the GROUP BY clause, so its syntax is easy to use.
The action of the ROLLUP function creates subtotals that roll up from the most detailed level to a grand total, following a grouping list specified in the ROLLUP function.
The ROLLUP function takes as its argument an ordered list of grouping columns.

Syntax
SELECT ... GROUP BY ROLLUP(grouping_column_reference_list);
Example:
SELECT Time, Region, Department, sum(Profit) AS Profit FROM sales GROUP BY ROLLUP(Time, Region, Dept);
[Example]

CUBE

The CUBE function takes a specified set of grouping columns and creates subtotals for all of the possible combinations.
If you have specified CUBE(time, region, department), the result set will include all the values that would be included in an equivalent ROLLUP function plus additional combinations.
ROLLUP and CUBE are simple extensions to the SELECT statement's GROUP BY clause.
CUBE is an extension similar to ROLLUP, enabling a single statement to calculate all possible combinations of subtotals.
CUBE can generate the information needed in cross-tab reports with a single query.
The data needed for cross-tabular reports can be generated with a single SELECT statement using the CUBE function.

SYNTAX : SELECT ... GROUP BY CUBE (grouping_column_reference_list)

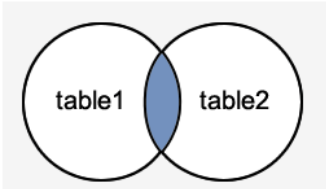
Example :
SELECT Time, Region, Department,sum(Profit) AS Profit FROM sales GROUP BY CUBE(Time, Region, Dept);[Example]

Oracle Joins

- Join is a query that is used to combine rows from two or more tables or views.
- It retrieves data from multiple tables and creates a new table.
- There may be at least one join condition either in the FROM clause or in the WHERE clause for joining two tables.
- It compares two columns from different tables and combines pair of rows, each containing one row from each table, for which join condition is true.

Types of Joins

- Inner Joins (Simple Join)
- Outer Joins
  - Left Outer Join (Left Join)
  - Right Outer Join (Right Join)
  - Full Outer Join (Full Join)
- Equijoins
- Self Joins
- Cross Joins (Cartesian Products)
- Antijoins
- Semi joins



1. Inner Join

Inner Join is the simplest and most common type of join. It is also known as simple join. It returns all rows from multiple tables where the join condition is met.

Syntax

SELECT columns FROM table1 INNER JOIN table2 ON table1.column = table2.column;

Example

Let's take an example to perform Inner Join on two tables "Suppliers" and "Order1".

Suppliers:

EDIT	SUPPLIER_ID	SUPPLIER_NAME	SUPPLIER_ADDRESS
	1	Bata shoes	Agra
	2	Kingfisher	Delhi
	3	VOJO	Lucknow

order1:

EDIT	ORDER_NUMBER	SUPPLIER_ID	CITY
	101	1	Allahabad
	102	2	Kanpur

This example will return all rows from "suppliers" and "order1" table where there is a matching supplier_id value in both the suppliers and order1 tables.

QUERY : SELECT suppliers.supplier_id, suppliers.supplier_name, order1.order_number FROM suppliers INNER JOIN order1 ON suppliers.supplier_id = order1.supplier_id;

Output

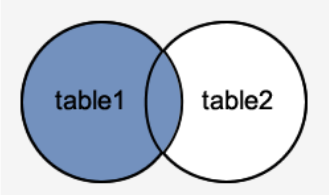
SUPPLIER_ID	SUPPLIER_NAME	ORDER_NUMBER
1	Bata shoes	101
2	Kingfisher	102

[Natural Join : It is a type of inner type that joins two or more tables based on the same column name and has the same data type present on both tables. **]**

2. OUTER JOIN

An outer join is similar to equijoin but it gets also the non-matched rows from the table.
It is categorized in Left Outer Join, Right Outer Join and Full Outer Join by Oracle 9i ANSI/ISO 1999 standard.

Left Outer Join : Left Outer Join returns all rows from the left (first) table specified in the ON condition and only those rows from the right (second) table where the join condition is met.



Syntax : SELECT columns FROM table1 LEFT [OUTER] JOIN table2
ON table1.column = table2.column;

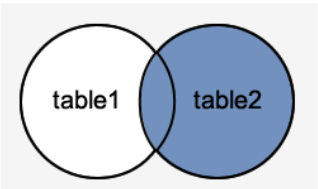
Example: we are performing left outer join on the already created tables 'suppliers' and 'order1'. The following example would return all records from table 'suppliers' and only those records from table 'order1' where the join fields are equal.

QUERY : SELECT suppliers.supplier_id, suppliers.supplier_name, order1.order_number
FROM suppliers LEFT OUTER JOIN order1 ON suppliers.supplier_id = order1.supplier_id;

Output

SUPPLIER_ID	SUPPLIER_NAME	ORDER_NUMBER
1	Bata shoes	101
2	Kingfisher	102
50	Flipkart	-
50	Apple	-
3	VOJO	-
21	Microsoft	-
21	ghaziabad	-
30	Google	-
28	Paris	-
22	Kingfisher	-

Right Outer Join : The Right Outer Join returns all rows from the right-hand table specified in the ON condition and only those rows from the other table where the join condition is met.



Syntax : SELECT columns FROM table1 RIGHT [OUTER] JOIN table2
ON table1.column = table2.column;

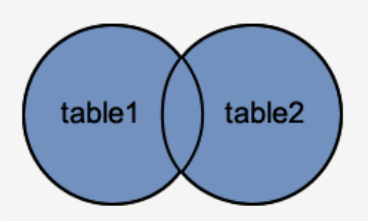
Example : In this example, we are performing right outer join on the already created tables 'suppliers' and 'order1'.
The following example would return all rows from the order1 table and only those rows from the suppliers table where the join condition is met.

QUERY : SELECT order1.order_number, order1.city, suppliers.supplier_name FROM suppliers
RIGHT OUTER JOIN order1 ON suppliers.supplier_id = order1.supplier_id;

Output

ORDER_NUMBER	CITY	SUPPLIER_NAME
101	Allahabad	Bata shoes
102	Kanpur	Kingfisher
105	Ghaziabad	-

Full Outer Join : The Full Outer Join returns all rows from the left hand table and right hand table. It places NULL where the join condition is not met.



Syntax : SELECT columns FROM table1 FULL [OUTER] JOIN table2 ON table1.column = table2.column;

Example : In this example, we are performing full outer join on the already created tables 'suppliers' and 'order1'.

The following example will return all rows from the 'suppliers' table and all rows from the 'order1' table and whenever the join condition is not met, it places the NULL value.

QUERY : SELECT suppliers.supplier_id, suppliers.supplier_name, order1.order_number FROM suppliers FULL OUTER JOIN order1 ON suppliers.supplier_id = order1.supplier_id;

Output :

SUPPLIER_ID	SUPPLIER_NAME	ORDER_NUMBER
1	Bata shoes	101
2	Kingfisher	102
50	Flipkart	-
50	Apple	-
3	VOJO	-
21	Microsoft	-
21	ghaziabad	-
30	Google	-
28	Paris	-
22	Kingfisher	-

3. Oracle SELF JOIN

Self Join is a specific type of Join. In Self Join, a table is joined with itself (Unary relationship).

A self join simply specifies that each rows of a table is combined with itself and every other row of the table.

Syntax

SELECT a.column_name, b.column_name
FROM table1 a, table1 b WHERE a.common_filed = b.common_field;

Example :

Let's take a table "customers".

Join this table using SELF JOIN as follows:

SELECT a.name, b.age, a.SALARY FROM CUSTOMERS a, CUSTOMERS b WHERE a.SALARY < b.SALARY;

EDIT	NAME	AGE	ADDRESS	SALARY
	Alex	24	NewYork	25000
	Pandian	32	Chennai	32000
	Lalu	45	Bihar	56000
	Bholu	19	Haridwar	12000

Output :

NAME	AGE	SALARY
Alex	32	25000
Alex	45	25000
Pandian	45	32000
Bholu	24	12000
Bholu	32	12000
Bholu	45	12000

WHAT IS SUB QUERY? EXPLAIN ITS TYPE.

- Query within a Query is called sub query.
- It is also known as nested query.
- The statement containing a sub query is called parent query.
- Typically sub query appear in the WHERE condition of SELECT statement
- There are 5 types of sub queries:
    - Single row sub query
    - Multiple row sub query
    - Multiple value sub query
    - Multiple column sub query
    - Correlated sub query

SINGLE ROW SUB QUERY:

- It returns a single value from table.
- The sub query result is than compare to the value of columns in the parent query
- A single row sub query is use a equality operator

EXAMPLE:
SELECT ENAME, SAL FROM EMP WHERE SAL = (SELECT MAX (SAL) FROM EMP);

MULTIPLE VALUE SUB QUERY:

- This returns more than one value to the parent query statements.
- IN operator this the most commonly use multiple row sub query.
- We can use exists, all or any operators.

EXAMPLE
SELECT ENAME, SAL, JOB FROM EMP WHERE SAL IN (SELECT MAX (SAL) FROM EMP GROUP BY JOB);

MULTIPLE COLUMN SUB QUERY:

- This returns more value from more than one column.
- A sub query is multiple columns where there is more than one column in select statement of sub query
- Multiple column sub query are generally used to compare column condition.

EXAMPLE:
SELECT ENAME, SAL, JOB FROM EMP WHERE (SAL, JOB) IN (SELECT MAX (SAL), JOB FROM EMP GROUP BY JOB);

MULTIPLE ROW SUB QUERY:

- This type of sub query returns more than one row of result from the sub query.

EXAMPLE:
SELECT ENAME, DEPTNO FROM EMP WHERE DEPTNO NOT IN (SELECT DEPTNO FROM EMP WHERE ENAME='SCOTT');

CORELATED SUB QUERY:

- This type of sub queries is performed when the sub query references a column from a table refer to in the parent statement.
- The parent statement can be a select update or delete

EXAMPLE:
SELECT EMPNO, ENAME, JOB, SAL, DEPTNO FROM emp e WHERE sal > (SELECT AVG(sal) FROM emp WHERE DEPTNO = e.DEPTNO) ORDER BY DEPTNO;

Operator to get combine result of multiple queries:

Minus: The Minus is an operator in Structured Query Language, which is used with two SELECT queries. This operator returns only unique records of the first table, not the common records of both tables. [Example]

- **Intersect:** The INTERSECT operator in SQL is used to retrieve the records that are identical/common between the result sets of two SELECT (tables) statements. [Example]
- **Union**: UNION is an SQL operator which combines the result of two or more SELECT queries and provides the single set in the output. [Example]

EXPLAIN BUILT IN FUNCTIONS OF ORACLE

ABS()	Returns positive value of negative numbers	SELECT ABS(-55) FROM DUAL;	55
COS()	Returns cos angles value.	SELECT COS(60) FROM DUAL;	-9.52413
SIN()	Returns sin angles value.	SELECT SIN(60) FROM DUAL;	-.3048106
TAN()	Returns tan angles value.	SELECT TAN(60) FROM DUAL;	-.32004039
SUM()	Returns sum of expression	SELECT SUM(SAL) FROM EMP;	29025
AVG()	Returns average of expression	SELECT AVG(SAL) FROM EMP;	2073.21429
MAX()	Returns maximum value of expression	SELECT MAX(SAL) FROM EMP;	5000
MIN()	Returns minimum value of expression	SELECT MIN(SAL) FROM EMP;	800
CEIL()	Returns the nearest highest integer values of an expression	SELECT CEIL(3.244) FROM DUAL;	4
FLOOR()	Returns the nearest highest integer values of an expression	SELECT FLOOR(3.545) FROM DUAL;	3
MOD()	Returns the reminder after dividing M with N	SELECT MOD(5,2) FROM DUAL;	1
POWER()	Returns the power of M with N	SELECT POWER(5,2) FROM DUAL;	25
LOG()	Returns the natural log of base M of N	SELECT LOG(10,1000) FORM DUAL;	2
ROUND()	Returns the round of values of given expression	SELECT ROUND(6.8985,2) FROM DUAL;	6.49
TRUNC()	Similar to ROUND().Returns the round of values of given expression	SELECT TRUNC(6.8985,0) FROM DUAL;	6.48
SQRT()	Returns the square root of specified number	SELECT SQRT(25) FROM DUAL;	5
LOWER()	Convert string in lowercase	SELECT LOWER('GEETANJALI') FROM DUAL;	geetanjali
UPPER()	Convert string in uppercase	SELECT UPPER('geetanjali') FROM DUAL;	GEETANJALI
INITCAP()	Convert each word of string in capital letter.	SELECT INITCAP('geetanjali college') FROM DUAL;	Geetanajli College
LENGTH()	Returns the length of a given string	SELECT LENGTH('GEETANJALI') FROM DUAL;	10
SUBSTR()	Returns substring from given string starting position P to N characters	SELECT SUBSTR('GEETANJALI',5,6) FROM DUAL;	ANJALI
INSTR()	Used to find a particular character in a string	SELECT INSTR('GEETANJALI','E') FROM DUAL;	2

REPLACE()	Replace a specified string with given string.	SELECT REPLACE('GEETANJALI SCHOOL','SCHOOL','COLLEGE') FROM DUAL;	GEETANJALI COLLEGE
RPAD()	Right pad given string with a given character to N number of character	SELECT RPAD('ABC',5,'*') FROM DUAL;	ABC**
LPAD()	Left pad given string with a given character to N number of character	SELECT LPAD('ABC',5,'*') FROM DUAL;	**ABC
LTRIM()	Remove the leading space before the string from left side.	SELECT LTRIM(' ABC ') FROM DUAL;	ABC
RTRIM()	Remove the space after the string from left side.	SELECT LTRIM(' ABC ') FROM DUAL;	ABC
CONCAT()	Used to merge to strings.	SELECT CONCATE('ABC','IS NAME') FROM DUAL;	ABC IS NAME
COUNT()	Returns the number of Rows in the query.	SELECT COUNT(ENAME) FORM EMP;	14
USER()	Returns the current logon users name	SELECT USER FROM EMP;	SCOTT
GREATEST()	Returns the highest value from given expression	SELECT GREATEST(10,20,30,45,60) FROM DUAL;	60
LEAST()	Returns the highest value from given expression	SELECT LEAST(10,20,30,45,60) FROM DUAL;	10
ASCII	Returns an ASCII code value of a character.	ASCII('A')	65
CHR	Converts a numeric value to its corresponding ASCII character.	CHR('65')	A
ADD_MONTHS	Add a number of months (n) to a date and return the same day which is n of months away.	ADD_MONTHS(DATE '2016-02-29', 1)	31-MAR-16
LAST_DAY	Gets the last day of the month of a specified date.	LAST_DAY(DATE '201602-01')	29-FEB-16
MONTHS_BETWEEN	Return the number of months between two dates.	MONTHS_BETWEEN(DATE '2017-07-01', DATE '2017-01-01')	6
NEXT_DAY	Get the first weekday that is later than a specified date.	NEXT_DAY(DATE '200001-01', 'SUNDAY')	02-JAN-00
SYSDATE	Return the current system date and time of the operating system where the Oracle Database resides.	SYSDATE	01-AUG-17
SYSTIMESTAMP	Return the system date and time that includes fractional seconds and time zone.	SELECT SYSTIMESTAMP FROM dual	01-AUG-17 01.33.57.929000000 PM -07:00
TO_CHAR	Convert a date which is in the character string to a DATE value.	TO_DATE('01 Jan 2017', 'DD MON YYYY') [example and access specifier]	01-JAN-17

TO_DURATION	Casts a string representation of a timestamp into a number of milliseconds so that it can be used as a duration.		
TO_INTEGER(Boolean)	Casts TRUE/FALSE to 1/0.		
TO_DOUBLE	Casts a string representation of an integer as a double.		
DECODE()	Compares expression with search value one by one if the expression does not match then any of search values returns the default value, if the default value is omitted than returns NULL Syntax: SELECT DECODE(expr,search value1,result1, search value2,result2, ….) AS <column name> FROM <tablename>;	SELECT DECODE (DEPTNO, 10,'ADMIN',20,'SALES', 30, 'PRODUCTION') AS DEPTNAME FROM EMP; SELECT DECODE (DEPTNO, 10,'ADMIN',20,'SALES') AS DEPTNAME FROM EMP;	

CREATE USER :

Use the CREATE USER statement to create and configure a database user, which is an account through which you can log in to the database, and to establish the means by which Oracle Database permits access by the user.

You can enable a user to connect to the database through a proxy application or application server.

SYNTAX :

```
CREATE USER username
IDENTIFIED BY password
[DEFAULT TABLESPACE tablespace]
[QUOTA {size | UNLIMITED} ON tablespace]
[PROFILE profile]
[PASSWORD EXPIRE]
[ACCOUNT {LOCK | UNLOCK}];
```

Here..

CREATE USER username:	Specify the name of the user to be created.
IDENTIFIED BY password:	Specify a password for the local user to use to log on to the database.
DEFAULT TABLESPACE:	Specify the tablespace of the objects such as tables and views that the user will create. [If you skip this clause, the user's objects will be stored in the database default tablespace if available, typically it is USERS tablespace; or the SYSTEM tablespace in case there is no database default tablespace.]
QUOTA	Use the QUOTA clause to specify the maximum amount of space the user can allocate in the tablespace.
PROFILE	Specify the profile you want to assign to the user. The profile limits the amount of database resources the user can use. If you omit this clause, then Oracle Database assigns the DEFAULT profile to the user.
PASSWORD EXPIRE	Specify PASSWORD EXPIRE if you want the user's password to expire. This setting forces the user or the DBA to change the password before the user can log in to the database.
ACCOUNT	Specify ACCOUNT LOCK to lock the user's account and disable access. Specify ACCOUNT UNLOCK to unlock the user's account and enable access to the account. The default is ACCOUNT UNLOCK.

[More Explanation]

```
Example : CREATE USER TEST IDENTIFIED BY "123";
```

CREATE ROLE: [MORE INFORMATION]

- A role is a group of privileges.
- Instead of granting individual privileges to users, you can group related privileges into a role and grant this role to users.
- Roles help manage privileges more efficiently.
- To create a new role, you use the CREATE ROLE statement.

SYNTAX:

```
CREATE ROLE role_name [ NOT IDENTIFIED |
IDENTIFIED {BY password | USING [schema.] package | EXTERNALLY | GLOBALLY};
```

role_name	The name of the new role that you are creating. This is how you will refer to the grouping of privileges.
NOT IDENTIFIED	It means that the role is immediately enabled. No password is required to enable the role.
IDENTIFIED	It means that a user must be authorized by a specified method before the role is enabled.
BY password	It means that a user must supply a password to enable the role.
USING package	It means that you are creating an application role - a role that is enabled only by applications using an authorized package.
EXTERNALLY	It means that a user must be authorized by an external service to enable the role. An external service can be an operating system or third-party service.
GLOBALLY	It means that a user must be authorized by the enterprise directory service to enable the role.

Data Control Language (DCL) Statement

DCL is used to control privileges in Database.

To perform any operation in the database, such as for creating tables, sequences or views, a user needs privileges.

1. Grant:

After creating a user, you need to decide which actions the user can do in the Oracle database. By definition, a privilege is a right to execute an SQL statement or a right to access an object of another user. Oracle defines two main types of privileges: system privileges and object privileges.

a. **System privileges:** System privileges determine what a user can do in the database. They mainly allow a user to add or modify schema objects in the database like creating tables, creating views, and removing tablespaces. The most important system privileges are: CREATE SESSION, CREATE TABLE, CREATE VIEW, CREATE PROCEDURE, SYSDBA, SYSOPER.

b. **Object privileges:** Object privileges decide how a user can access the data in the database. The object privileges apply to rows in tables or views. Here are some common object privileges: INSERT, UPDATE, DELETE, INDEX, EXECUTE. To grant one or more privileges to a user, you use the GRANT statement

The GRANT statement assigns one or more privileges to a specific user. The following illustrates the basic syntax of the GRANT statement:

```
GRANT {system_privileges | object_privileges }
TO user [WITH ADMIN OPTION]
```

In this syntax: First, specify the system or object privileges that you want to assign to a user after the GRANT keyword. If you assign more than one privilege, you use a comma-separated list of privileges. Second, specify the user that receives the privileges after the TO keyword.

Example : GRANT all ON emp TO warehouse_user; Where emp is table name and warehouse_user is a role name.

[Extra Information]

Revoke: Use the REVOKE statement to remove privileges from a specific user or role, or from all users, to perform actions on database objects. Here is the basic syntax of the Oracle REVOKE statement:

```
REVOKE {system_privilege | object_privilege } FROM user;
```

In this syntax: First, specify the system or object privileges that you want to revoke from the user. Second, specify the user from which you want to revoke the privileges. [Extra Info]

What is transaction?

A transaction is a logical unit of work that contains one or more SQL statements.
A transaction begins with the first executable SQL statement.
A transaction ends when it is committed or rolled back, either explicitly with a COMMIT or ROLLBACK statement or implicitly when a DDL statement is issued.
Transaction Control Language (TCL) commands are used to manage transactions in the database.

Commit: COMMIT command is used to permanently save any transaction into the database. When we use any DML command like INSERT, UPDATE or DELETE, the changes made by these commands are not permanent, until the current session is closed, the changes made by these commands can be rolled back. To avoid that, we use the COMMIT command to mark the changes as permanent. Following is commit command's syntax:

```
COMMIT;
```

To execute a COMMIT automatically whenever an INSERT, UPDATE or DELETE command is executed, you can set the AUTOCOMMIT environment variable as −

```
SET AUTOCOMMIT ON;
```

You can turn-off the auto commit mode using the following command −

```
SET AUTOCOMMIT OFF;
```

Rollback: This command restores the database to last committed state. It is also used with SAVEPOINT command to jump to a savepoint in an ongoing transaction. If we have used the UPDATE command to make some changes into the database, and realise that those changes were not required, then we can use the ROLLBACK command to rollback those changes, if they were not committed using the COMMIT command. Following is rollback command's syntax:

```
ROLLBACK;
ROLLBACK TO savepoint_name;
```

Savepoint: Savepoints are sort of markers that help in splitting a long transaction into smaller units by setting some checkpoints. By setting savepoints within a long transaction, you can roll back to a checkpoint if required. This is done by issuing the SAVEPOINT command. Following is savepoint command's syntax:

```
SAVEPOINT savepoint_name;
```

In short, using this command we can name the different states of our data in any table and then rollback to that state using the ROLLBACK command whenever required.

Example :

```
INSERT INTO CUSTOMERS (ID,NAME,AGE,ADDRESS,SALARY)
VALUES (7, 'Rajnish', 27, 'HP', 9500.00 );

INSERT INTO CUSTOMERS (ID,NAME,AGE,ADDRESS,SALARY)
VALUES (8, 'Riddhi', 21, 'WB', 4500.00 );
SAVEPOINT sav1;

UPDATE CUSTOMERS
SET SALARY = SALARY + 1000;
ROLLBACK TO sav1;

UPDATE CUSTOMERS
SET SALARY = SALARY + 1000
```

```
WHERE ID = 7;
UPDATE CUSTOMERS
SET SALARY = SALARY + 1000
WHERE ID = 8;

COMMIT;
```

Extra Explanation

An Oracle database consists of one or more logical storage units called **tablespaces**, which collectively store all of the database's data.
Each tablespace in an Oracle database consists of one or more files called datafiles, which are physical structures that conform to the operating system in which Oracle is running.

Practical Example :

Some page setup in sqlplus :

```
select id||' '||name||' '||city from student3a;
select empno||' '||sal||' '||job from emp;
SET LINESIZE 32000;
SET PAGESIZE 40000;
```

SQL Constraints :

SQL NOT NULL on CREATE TABLE

```
CREATE TABLE Persons (ID int NOT NULL, LastName varchar(255) NOT NULL, FirstName varchar(255) NOT NULL, Age int );
```

SQL NOT NULL on ALTER TABLE

```
ALTER TABLE student3a MODIFY id number(3) NOT NULL;
```

SQL UNIQUE Constraint on CREATE TABLE

```
CREATE TABLE Persons (
   ID int NOT NULL UNIQUE,
   LastName varchar(255) NOT NULL,
   FirstName varchar(255),
   Age int
);

CREATE TABLE Persons (
   ID int NOT NULL,
   LastName varchar(255) NOT NULL,
   FirstName varchar(255),
   Age int,
   CONSTRAINT UC_Person UNIQUE (ID,LastName)
);
```

SQL UNIQUE Constraint on ALTER TABLE

```
ALTER TABLE Persons
ADD CONSTRAINT UC_Person UNIQUE (ID,LastName);
```

DROP a UNIQUE Constraint

```
ALTER TABLE Persons
DROP CONSTRAINT UC_Person;
```

SQL PRIMARY KEY Constraint

```
CREATE TABLE Persons (
   ID int NOT NULL,
   LastName varchar(255) NOT NULL,
   FirstName varchar(255),
   Age int,
   PRIMARY KEY (ID)
);

CREATE TABLE Persons (
   ID int NOT NULL,
   LastName varchar(255) NOT NULL,
   FirstName varchar(255),
   Age int,
   CONSTRAINT PK_Person PRIMARY KEY (ID,LastName)
);
```

Note : In the example above there is only ONE PRIMARY KEY (PK_Person). However, the VALUE of the primary key is made up of TWO COLUMNS (ID + LastName).

SQL PRIMARY KEY on ALTER TABLE

```
ALTER TABLE Persons
ADD CONSTRAINT PK_Person PRIMARY KEY (ID,LastName);
```

DROP a PRIMARY KEY Constraint

```
ALTER TABLE Persons
DROP PRIMARY KEY;

ALTER TABLE Persons
DROP CONSTRAINT PK_Person;
```

SQL FOREIGN KEY Constraint

Persons Table

PersonID	**LastName**	**FirstName**	**Age**
1	Hansen	Ola	30
2	Svendson	Tove	23
3	Pettersen	Kari	20

Orders Table

OrderID	OrderNumber	PersonID
1	77895	3
2	44678	3
3	22456	2
4	24562	1

SQL FOREIGN KEY on CREATE TABLE

***ALTER TABLE persons RENAME COLUMN id to personid;

```
CREATE TABLE Orders (
   OrderID int NOT NULL PRIMARY KEY,
   OrderNumber int NOT NULL,
   OrderPersonID int  REFERENCES Persons(PersonID)
);
CREATE TABLE Orders (
   OrderID int NOT NULL,
   OrderNumber int NOT NULL,
   PersonID int,
   PRIMARY KEY (OrderID),
   CONSTRAINT FK_PersonOrder FOREIGN KEY (PersonID)
   REFERENCES Persons(PERSONID)
);
```

SQL FOREIGN KEY on ALTER TABLE

```
ALTER TABLE Orders
ADD FOREIGN KEY (PersonID) REFERENCES Persons(PersonID);
ALTER TABLE Orders
ADD CONSTRAINT FK_PersonOrder
FOREIGN KEY (PersonID) REFERENCES Persons(PersonID);
```

DROP a FOREIGN KEY Constraint

```
ALTER TABLE Orders
DROP CONSTRAINT FK_PersonOrder;
```

SQL CHECK Constraint

```
CREATE TABLE Persons (
   ID int NOT NULL,
   LastName varchar(255) NOT NULL,
   FirstName varchar(255),
   Age int,
   CHECK (Age>=18)
);
CREATE TABLE Persons (
   ID int NOT NULL,
   LastName varchar(255) NOT NULL,
   FirstName varchar(255),
   Age int CHECK (Age>=18)
);
CREATE TABLE Persons (
   ID int NOT NULL,
   LastName varchar(255) NOT NULL,
   FirstName varchar(255),
   Age int,
   City varchar(255),
   CONSTRAINT CHK_Person CHECK (Age>=18 AND City='Sandnes')
);
```

SQL CHECK on ALTER TABLE

```
ALTER TABLE Persons
ADD CHECK (Age>=18);
ALTER TABLE Persons
ADD CONSTRAINT CHK_PersonAge CHECK (Age>=18 AND City='Sandnes');
```

DROP a CHECK Constraint

```
ALTER TABLE Persons
DROP CHECK CHK_PersonAge;
```

SQL DEFAULT Constraint

```
CREATE TABLE Persons (
   ID int NOT NULL,
   LastName varchar(255) NOT NULL,
   FirstName varchar(255),
   Age int,
   City varchar(255) DEFAULT 'Surat'
);

CREATE TABLE Orders (
   ID int NOT NULL,
   OrderNumber int NOT NULL,
   OrderDate date DEFAULT GETDATE()
);
```

SQL DEFAULT on ALTER TABLE

```
ALTER TABLE Persons modify City varchar2(20) DEFAULT 'Surat';
```

DROP a DEFAULT Constraint

```
ALTER TABLE Persons
ALTER City DROP DEFAULT;

ALTER TABLE Persons
ALTER COLUMN City DROP DEFAULT;

ALTER TABLE Persons
ALTER COLUMN City DROP DEFAULT;
```